SmartLife ESP32 Home Automation

Arduino .ino source code • formatted for reading and printing

Source file: SmartLife ESP32 Home Automation.ino
Lines of code: 1,728

```
/*
  Smart Life with ESP32
  ------------------------------------------------------------
  Combines the 4 uploaded sketches (PIR, Gas, LDR, Relay web control)
  into one web dashboard. The sensor logic is the same as your files.
  Wiring (same as before):
    Relay IN1 -&gt; GPIO 16      Relay IN3 -&gt; GPIO 18      Relay IN4 -&gt; GPIO 23
    PIR OUT   -&gt; GPIO 27      LDR AO    -&gt; GPIO 35      Gas AO    -&gt; GPIO 34
    Buzzer +  -&gt; GPIO 25      Blue LED  -&gt; GPIO 2 (onboard)
  Sensor -&gt; output (only while that sensor is switched ON in the page):
    PIR -&gt; Relay 3 (2 sec hold)       LDR -&gt; Relay 4 (dark = ON)
    Gas -&gt; Buzzer + LED (3 sec clear delay)
  While a sensor is OFF, its relay is controlled by the buttons / timer.
  Login:  user = admin   first password = smartlife
          (change the password later on the Settings tab)
  Timer uses internet time (NTP) set to India time (IST).
  Added in this version:
    - Telegram gas alert (set the bot token + chat IDs on the Settings tab)
    - Event log (Log tab: gas, motion, light, relay, sign-in events)
    - Change password (Settings tab)
*/
#include &lt;WiFi.h&gt;
#include &lt;WebServer.h&gt;
#include &lt;Preferences.h&gt;
#include &lt;WiFiClientSecure.h&gt;
#include &lt;HTTPClient.h&gt;
#include &lt;time.h&gt;
// ---------- WiFi Credentials ----------
const char* ssid = "Moto edge 70 pro";
const char* password = "Y12345678";
// ---------- Optional fixed IP (set to 1 and match your router) ----------
#define USE_STATIC_IP 0
IPAddress local_IP(192, 168, 1, 150);
IPAddress gateway(192, 168, 1, 1);
IPAddress subnet(255, 255, 255, 0);
IPAddress dns1(8, 8, 8, 8);
// ---------- Login ----------
const char* LOGIN_USER = "smartlife";
const char* LOGIN_PASS = "slesp32";   // first-time password. After you change it on the
                                      // Settings tab, the new one is saved in flash.
// Forgot the new password? Set this to 1, upload once, sign in with the
// password above, then set it back to 0 and upload again.
#define RESET_PASSWORD 0
// ---------- Time zone: IST = UTC+5:30 ----------
const long GMT_OFFSET_SEC = 19800;
// ---------- Pins ----------
#define RELAY1      16   // IN1
#define RELAY3      18   // IN3
#define RELAY4      23   // IN4
#define PIR_PIN     27
#define LDR_PIN     35
#define GAS_PIN     34
#define BUZZER_PIN  25
#define LED_PIN     2
// Most relay modules are ACTIVE LOW (LOW = ON, HIGH = OFF)
bool ACTIVE_LOW = true;
WebServer server(80);
Preferences prefs;
String loginPass;                  // current password (saved in flash)
String tgToken, tgIds;             // Telegram bot token and chat IDs (saved in flash)
bool tgOn = true;                  // Telegram alerts ON/OFF
SemaphoreHandle_t cfgMux;          // protects the Telegram settings between tasks
// ================================================================
//  Relays  (index 0 = Relay 1, 1 = Relay 3, 2 = Relay 4)
// ================================================================
const int relayPin[3] = {RELAY1, RELAY3, RELAY4};
const int relayId[3]  = {1, 3, 4};
bool manualOn[3]      = {false, false, false};
int  lastOut[3]       = {-1, -1, -1};
void setRelay(int pin, bool on) {
  if (ACTIVE_LOW) {
    digitalWrite(pin, on ? LOW : HIGH);
  } else {
    digitalWrite(pin, on ? HIGH : LOW);
  }
}
// ================================================================
//  Event log (last 60 events, shown on the Log tab)
// ================================================================
#define LOG_MAX 60
enum { K_SYS, K_GAS, K_MOTION, K_LIGHT, K_RELAY, K_SENSOR, K_LOGIN, K_TG };
struct Ev { time_t t; uint32_t ms; uint8_t k; char m[48]; };
Ev evs[LOG_MAX];
int evHead = 0, evCount = 0;
uint32_t evTotal = 0;          // goes up on every event (the page uses it to know when to refresh)
bool timeSynced = false;
bool relayOwned(int i);        // defined below
void logEvent(uint8_t k, const char *m) {
  time_t n = time(nullptr);
  Ev &amp;e = evs[evHead];
```

```
  e.t = (n &gt; 1483228800) ? n : 0;   // 0 = clock not synced yet (fixed once NTP works)
  e.ms = millis();
  e.k = k;
  strlcpy(e.m, m, sizeof(e.m));
  evHead = (evHead + 1) % LOG_MAX;
  if (evCount &lt; LOG_MAX) evCount++;
  evTotal++;
}
String evTime(int idx) {
  char b[24];
  if (evs[idx].t) {
    struct tm t;
    time_t tt = evs[idx].t;
    localtime_r(&amp;tt, &amp;t);
    strftime(b, sizeof(b), "%d %b %H:%M:%S", &amp;t);
  } else {
    snprintf(b, sizeof(b), "up %lus", (unsigned long)(evs[idx].ms / 1000));
  }
  return String(b);
}
// events logged before the clock synced get their real time once it does
void checkSync() {
  if (timeSynced) return;
  time_t n = time(nullptr);
  if (n &lt;= 1483228800) return;
  timeSynced = true;
  for (int i = 0; i &lt; evCount; i++)
    if (!evs[i].t) evs[i].t = n - (millis() - evs[i].ms) / 1000;
  logEvent(K_SYS, "Clock synced");
}
// change a relay's manual state and log it (only when it really changes)
void setManual(int i, bool on, const char *src) {
  if (manualOn[i] == on) return;
  manualOn[i] = on;
  char b[48];
  snprintf(b, sizeof(b), "Relay %d %s (%s)", relayId[i], on ? "ON" : "OFF",
           relayOwned(i) ? "timer, sensor in control" : src);
  logEvent(K_RELAY, b);
}
// ====================================================================
//  Sensors  (logic copied from your pir_only / ldr_relay4 / gas_alert files)
// ====================================================================
bool pirEn = true, ldrEn = true, gasEn = true;   // sensor ON/OFF buttons
// previous states, used to log changes
int prevGas = 0, prevLdr = 0;
bool prevPir = false, motionLogged = false;
unsigned long lastMotionLog = 0;
// ---- PIR ----
const unsigned long HOLD_TIME = 2000;   // 2 seconds
unsigned long motionStart = 0;
bool holding = false;
// ---- LDR ----
int ldrState = 0;                // 0 = light, 1 = dark
unsigned long lightSince = 0;
bool lightCandidate = false;
int ldrRaw = 0;
// ---- Gas ----
int GAS_ON_THRESHOLD  = 2800;    // raw value above this = gas alert
int GAS_OFF_THRESHOLD = 2600;    // raw value below this = clear
int gasState = 0;                // 0 = no gas, 1 = gas alert
unsigned long clearSince = 0;
bool clearCandidate = false;
int gasRaw = 0;
// average of 20 samples (keeps the web server answering in between)
int readSmoothed(int pin) {
  long total = 0;
  for (int i = 0; i &lt; 20; i++) {
    total += analogRead(pin);
    delay(2);
    server.handleClient();
  }
  return total / 20;
}
void pirTick() {
  if (!pirEn) { holding = false; return; }
  // PIR (2 sec hold) -&gt; Relay3
  if (holding &amp;&amp; millis() - motionStart &gt;= HOLD_TIME) {
    holding = false;
    Serial.println("Ready for next detection");
  }
  if (!holding &amp;&amp; digitalRead(PIR_PIN) == HIGH) {
    holding = true;
    motionStart = millis();
    Serial.println("Motion detected");
  }
}
void ldrTick() {
  if (!ldrEn) { ldrState = 0; lightCandidate = false; return; }
  int raw = readSmoothed(LDR_PIN);
  ldrRaw = raw;
```

```
  bool rawDark = (raw &gt; 770);
  bool rawLight = (raw &lt; 670);
  // Dark is detected instantly (checked every 0.2 sec).
  if (rawDark) {
    ldrState = 1;
    lightCandidate = false;
  }
  // Light only takes effect after it holds steady for 3 seconds.
  if (ldrState == 1 &amp;&amp; rawLight) {
    if (!lightCandidate) {
      lightCandidate = true;
      lightSince = millis();
    }
    if (millis() - lightSince &gt;= 3000) {
      ldrState = 0;
      lightCandidate = false;
    }
  } else if (!rawDark) {
    lightCandidate = false;
  }
}
void gasTick() {
  if (!gasEn) { gasState = 0; clearCandidate = false; return; }
  int raw = readSmoothed(GAS_PIN);
  gasRaw = raw;
  bool rawGas   = (raw &gt; GAS_ON_THRESHOLD);
  bool rawClear = (raw &lt; GAS_OFF_THRESHOLD);
  // Gas is detected instantly (checked every 0.2 sec).
  if (rawGas) {
    gasState = 1;
    clearCandidate = false;
  }
  // Clear only takes effect after it holds steady for 3 seconds.
  if (gasState == 1 &amp;&amp; rawClear) {
    if (!clearCandidate) {
      clearCandidate = true;
      clearSince = millis();
    }
    if (millis() - clearSince &gt;= 3000) {
      gasState = 0;
      clearCandidate = false;
    }
  } else if (!rawGas) {
    clearCandidate = false;
  }
}
// ---------- outputs ----------
bool relayOwned(int i) {           // true = a sensor is driving this relay
  return (i == 1 &amp;&amp; pirEn) || (i == 2 &amp;&amp; ldrEn);
}
bool relayEffective(int i) {
  if (i == 1 &amp;&amp; pirEn) return holding;          // PIR -&gt; Relay 3
  if (i == 2 &amp;&amp; ldrEn) return ldrState == 1;    // LDR -&gt; Relay 4
  return manualOn[i];                          // buttons / timer
}
void applyRelays() {
  for (int i = 0; i &lt; 3; i++) {
    bool s = relayEffective(i);
    if (lastOut[i] != (int)s) {
      setRelay(relayPin[i], s);
      lastOut[i] = s;
    }
  }
}
void applyOutputs() {
  digitalWrite(BUZZER_PIN, gasState == 1 ? HIGH : LOW);
  digitalWrite(LED_PIN, (holding || gasState == 1) ? HIGH : LOW);
}
// ======================================================================
//  Timers (daily ON / OFF time for each relay, saved in flash)
// ======================================================================
struct Sched { bool en; int onMin; int offMin; };
Sched sched[3] = {{false, 480, 1080}, {false, 480, 1080}, {false, 480, 1080}};
int lastMinute = -1;
void loadSched() {
  prefs.begin("smartlife", false);
  for (int i = 0; i &lt; 3; i++) {
    sched[i].en     = prefs.getBool(("e" + String(i)).c_str(), false);
    sched[i].onMin  = prefs.getShort(("n" + String(i)).c_str(), 480);
    sched[i].offMin = prefs.getShort(("f" + String(i)).c_str(), 1080);
  }
  prefs.end();
}
void saveSched(int i) {
  prefs.begin("smartlife", false);
  prefs.putBool(("e" + String(i)).c_str(), sched[i].en);
  prefs.putShort(("n" + String(i)).c_str(), sched[i].onMin);
  prefs.putShort(("f" + String(i)).c_str(), sched[i].offMin);
  prefs.end();
}
```

```
bool nowTime(struct tm &amp;t) {
  time_t n = time(nullptr);
  localtime_r(&amp;n, &amp;t);
  return t.tm_year &gt; (2016 - 1900);   // false until NTP has synced
}
void scheduleTick() {
  checkSync();
  struct tm t;
  if (!nowTime(t)) return;
  int nowMin = t.tm_hour * 60 + t.tm_min;
  if (nowMin == lastMinute) return;   // act once per minute
  lastMinute = nowMin;
  for (int i = 0; i &lt; 3; i++) {
    if (!sched[i].en) continue;
    if (nowMin == sched[i].offMin) setManual(i, false, "timer");
    if (nowMin == sched[i].onMin)  setManual(i, true, "timer");
  }
  applyRelays();
}
// ============================================================================
//  Login (session cookie, up to 4 devices at once)
// ============================================================================
#define MAX_SESS 4
String sess[MAX_SESS];
uint8_t sessNext = 0;
String newToken() {
  char b[33];
  for (int i = 0; i &lt; 4; i++) snprintf(b + i * 8, 9, "%08X", (unsigned)esp_random());
  b[32] = 0;
  return String(b);
}
String cookieToken() {
  if (!server.hasHeader("Cookie")) return "";
  String c = server.header("Cookie");
  int i = c.indexOf("SLTOK=");
  if (i &lt; 0) return "";
  i += 6;
  int e = c.indexOf(';', i);
  String t = (e &lt; 0) ? c.substring(i) : c.substring(i, e);
  t.trim();
  return t;
}
bool authed() {
  String t = cookieToken();
  if (t.length() != 32) return false;
  for (int k = 0; k &lt; MAX_SESS; k++) if (sess[k] == t) return true;
  return false;
}
// ============================================================================
//  Telegram alerts (sent from a background task so the page never waits)
// ============================================================================
struct TgMsg { char text[220]; };
QueueHandle_t tgQ;
volatile int tgCode = 0;            // 0 = nothing sent yet, 1 = not set up, 200 = OK, other = error
volatile uint32_t tgAtMs = 0;       // when the last send finished
unsigned long lastGasAlert = 0;
bool haveAlerted = false, gasAlertSent = false;
#define TG_MIN_GAP 60000UL          // at least 60 sec between two gas alerts
String urlEncode(const char *s) {
  String o;
  char b[4];
  for (; *s; s++) {
    unsigned char c = (unsigned char)*s;
    if (isalnum(c) || c == '-' || c == '_' || c == '.' || c == '~') {
      o += (char)c;
    } else {
      snprintf(b, sizeof(b), "%%%02X", c);
      o += b;
    }
  }
  return o;
}
void tgSendAll(const char *text) {
  char tok[64], ids[100];
  xSemaphoreTake(cfgMux, portMAX_DELAY);
  strlcpy(tok, tgToken.c_str(), sizeof(tok));
  strlcpy(ids, tgIds.c_str(), sizeof(ids));
  xSemaphoreGive(cfgMux);
  if (!tok[0] || !ids[0]) { tgCode = 1; tgAtMs = millis(); return; }
  String msg = urlEncode(text);
  int result = 200;
  char *sv = nullptr;
  // one message to every chat ID (separate IDs with commas)
  for (char *id = strtok_r(ids, ", ;", &amp;sv); id; id = strtok_r(nullptr, ", ;", &amp;sv)) {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String url = String("https://api.telegram.org/bot") + tok + "/sendMessage?chat_id=" + urlEncode(id) +
"&amp;text=" + msg;
    int code = -1;
```

```
    if (http.begin(client, url)) {
      http.setTimeout(8000);
      code = http.GET();
      http.end();
    }
    if (code != 200) result = code;
  }
  tgCode = result;
  tgAtMs = millis();
}
void tgTask(void *) {
  TgMsg m;
  for (;;) {
    if (xQueueReceive(tgQ, &amp;m, portMAX_DELAY) == pdTRUE) tgSendAll(m.text);
  }
}
void tgQueue(const char *text) {      // always sends (used by the test button)
  TgMsg m;
  strlcpy(m.text, text, sizeof(m.text));
  xQueueSend(tgQ, &amp;m, 0);
}
void tgNotify(const char *text) {     // sends only when alerts are ON
  if (tgOn) tgQueue(text);
}
// ====================================================================
//  Universal IR Remote (AC / TV / Projector) — learn once, replay anytime
//  ------------------------------------------------------------------
//  Wiring: IR receiver (TSOP1738) OUT -&gt; GPIO 21, IR LED (through your
//  transistor/resistor) -&gt; GPIO 22 — same pins as your PIR-test build.
//
//  Pairing happens in the Serial Monitor (115200 baud, line ending set
//  to "Newline"). Type one command per line:
//    pair ac  &lt;name&gt;     e.g. pair ac power   then point the AC remote
//                            at the receiver and press the button
//                            within 15 seconds
//    pair tv  &lt;name&gt;     e.g. pair tv vol_up
//    pair proj &lt;name&gt;    e.g. pair proj source
//    list ac | list tv | list proj      - show what is paired
//    send ac power                      - test a paired button
//    clear ac power                     - remove one button
//    clear ac all                       - wipe a whole device
//    cancel                             - stop a pairing in progress
//
//  Paired buttons appear automatically on http://&lt;device-ip&gt;/ir - no
//  code changes needed when you add a new one.
// ====================================================================
#include &lt;IRrecv.h&gt;
#include &lt;IRsend.h&gt;
#include &lt;IRutils.h&gt;
#define IR_RECV_PIN 21
#define IR_SEND_PIN 22
#define IR_CAPTURE_BUF 1500   // AC codes run much longer than TV/Projector codes
IRrecv irRecv(IR_RECV_PIN, IR_CAPTURE_BUF, 50, true);   // 50ms gap-timeout: AC remotes pause longer between
frames than TV remotes do
IRsend irSend(IR_SEND_PIN);
decode_results irResults;
enum { DEV_AC = 0, DEV_TV = 1, DEV_PROJ = 2, DEV_COUNT = 3 };
const char* const irDevKey[DEV_COUNT]   = {"ac", "tv", "proj"};
const char* const irDevLabel[DEV_COUNT] = {"AC", "TV", "Projector"};
// ---- AC temperature display only (the ESP32 has no way to read the AC's
// real state - this just remembers the last temperature button you sent,
// so the /ir page can show it) ----
int acTemp = 24;
const int AC_TEMP_MIN = 16, AC_TEMP_MAX = 30;
void saveAcTemp() {
  prefs.begin("smartlife", false);
  prefs.putInt("actemp", acTemp);
  prefs.end();
}
#define IR_MAX_BTN 16
#define IR_MAX_RAW 800   // longest raw code accepted (AC codes can run 500-700+ entries); actual RAM used is
per-button, not reserved upfront
struct IrButton {
  bool used;
  char name[20];
  bool isRaw;
  decode_type_t protocol;
  uint64_t value;
  uint16_t bits;
  uint16_t rawLen;
  uint16_t *rawData;   // allocated on the heap only for buttons that are actually raw
};
IrButton irBtn[DEV_COUNT][IR_MAX_BTN];   // small now - each slot is just a few bytes plus one pointer
bool irPairing = false;
int  irPairDev = -1;
char irPairName[20];
unsigned long irPairStart = 0;
const unsigned long IR_PAIR_TIMEOUT = 15000;
String irLine;
int irDevIndex(String s) {
```

```
  s.trim(); s.toLowerCase();
  for (int i = 0; i &lt; DEV_COUNT; i++) if (s == irDevKey[i]) return i;
  return -1;
}
int irFindButton(int dev, const char* name) {
  for (int i = 0; i &lt; IR_MAX_BTN; i++)
    if (irBtn[dev][i].used &amp;&amp; strcmp(irBtn[dev][i].name, name) == 0) return i;
  return -1;
}
int irFreeSlot(int dev, const char* name) {
  int existing = irFindButton(dev, name);
  if (existing &gt;= 0) return existing;      // re-pairing the same name overwrites it
  for (int i = 0; i &lt; IR_MAX_BTN; i++) if (!irBtn[dev][i].used) return i;
  return -1;
}
// ---- flash storage: one compact blob per device, sized to what's actually paired ----
const char* irPrefKey(int dev) {
  static const char* keys[DEV_COUNT] = {"ir_ac", "ir_tv", "ir_proj"};
  return keys[dev];
}
void irSaveDevice(int dev) {
  size_t sz = 1;   // button count byte
  for (int i = 0; i &lt; IR_MAX_BTN; i++) {
    if (!irBtn[dev][i].used) continue;
    sz += 1 + strlen(irBtn[dev][i].name) + 1 + 4 + 8 + 2 + 2;
    if (irBtn[dev][i].isRaw) sz += (size_t)irBtn[dev][i].rawLen * 2;
  }
  uint8_t *buf = (uint8_t*)malloc(sz);
  if (!buf) { Serial.println(F("[IR] Not enough RAM to save right now.")); return; }
  size_t p = 0;
  uint8_t count = 0;
  for (int i = 0; i &lt; IR_MAX_BTN; i++) if (irBtn[dev][i].used) count++;
  buf[p++] = count;
  for (int i = 0; i &lt; IR_MAX_BTN; i++) {
    if (!irBtn[dev][i].used) continue;
    IrButton &amp;b = irBtn[dev][i];
    uint8_t nlen = strlen(b.name);
    buf[p++] = nlen;
    memcpy(buf + p, b.name, nlen); p += nlen;
    buf[p++] = b.isRaw ? 1 : 0;
    uint32_t proto = (uint32_t)b.protocol; memcpy(buf + p, &amp;proto, 4); p += 4;
    memcpy(buf + p, &amp;b.value, 8); p += 8;
    memcpy(buf + p, &amp;b.bits, 2); p += 2;
    memcpy(buf + p, &amp;b.rawLen, 2); p += 2;
    if (b.isRaw) { memcpy(buf + p, b.rawData, (size_t)b.rawLen * 2); p += (size_t)b.rawLen * 2; }
  }
  prefs.begin("smartlife", false);
  bool ok = (prefs.putBytes(irPrefKey(dev), buf, sz) == sz);
  prefs.end();
  free(buf);
  if (!ok) Serial.println(F("[IR] Save failed - flash may be full. Try clearing an old button."));
}
void irLoadDevice(int dev) {
  for (int i = 0; i &lt; IR_MAX_BTN; i++) irBtn[dev][i].used = false;
  prefs.begin("smartlife", true);
  size_t sz = prefs.getBytesLength(irPrefKey(dev));
  if (sz == 0) { prefs.end(); return; }
  uint8_t *buf = (uint8_t*)malloc(sz);
  if (!buf) { prefs.end(); return; }
  prefs.getBytes(irPrefKey(dev), buf, sz);
  prefs.end();
  size_t p = 0;
  if (p &gt;= sz) { free(buf); return; }
  uint8_t count = buf[p++];
  for (uint8_t n = 0; n &lt; count &amp;&amp; p &lt; sz; n++) {
    if (n &gt;= IR_MAX_BTN) break;
    IrButton &amp;b = irBtn[dev][n];
    uint8_t nlen = buf[p++];
    if (nlen &gt;= sizeof(b.name)) nlen = sizeof(b.name) - 1;
    memcpy(b.name, buf + p, nlen); b.name[nlen] = 0; p += nlen;
    b.isRaw = buf[p++] != 0;
    uint32_t proto; memcpy(&amp;proto, buf + p, 4); p += 4; b.protocol = (decode_type_t)proto;
    memcpy(&amp;b.value, buf + p, 8); p += 8;
    memcpy(&amp;b.bits, buf + p, 2); p += 2;
    memcpy(&amp;b.rawLen, buf + p, 2); p += 2;
    if (b.rawLen &gt; IR_MAX_RAW) b.rawLen = IR_MAX_RAW;
    if (b.isRaw) {
      b.rawData = (uint16_t*)malloc((size_t)b.rawLen * 2);
      if (b.rawData) memcpy(b.rawData, buf + p, (size_t)b.rawLen * 2);
      p += (size_t)b.rawLen * 2;
    } else {
      b.rawData = nullptr;
    }
    b.used = true;
  }
  free(buf);
}
void irClearDevice(int dev) {
  for (int i = 0; i &lt; IR_MAX_BTN; i++) {
    if (irBtn[dev][i].rawData) { free(irBtn[dev][i].rawData); irBtn[dev][i].rawData = nullptr; }
```

```
    irBtn[dev][i].used = false;
  }
  irSaveDevice(dev);
}
void irDeleteButton(int dev, const char* name) {
  int i = irFindButton(dev, name);
  if (i &lt; 0) return;
  if (irBtn[dev][i].rawData) { free(irBtn[dev][i].rawData); irBtn[dev][i].rawData = nullptr; }
  irBtn[dev][i].used = false;
  irSaveDevice(dev);
}
void irPrintList(int dev) {
  Serial.print(F("[IR] ")); Serial.print(irDevLabel[dev]); Serial.println(F(" buttons:"));
  bool any = false;
  for (int i = 0; i &lt; IR_MAX_BTN; i++) {
    if (!irBtn[dev][i].used) continue;
    any = true;
    Serial.print(F("  - ")); Serial.print(irBtn[dev][i].name);
    Serial.println(irBtn[dev][i].isRaw ? F("  (raw)") : F("  (protocol)"));
  }
  if (!any) Serial.println(F("  (none paired yet)"));
}
void irPrintHelp() {
  Serial.println(F("[IR] Commands:"));
  Serial.println(F("  pair ac|tv|proj &lt;name&gt;   - learn a button, then press the remote"));
  Serial.println(F("  list ac|tv|proj          - show paired buttons"));
  Serial.println(F("  send ac|tv|proj &lt;name&gt;   - test-send a paired button"));
  Serial.println(F("  clear ac|tv|proj &lt;name&gt;  - remove one button"));
  Serial.println(F("  clear ac|tv|proj all     - wipe that device"));
  Serial.println(F("  cancel                   - stop a pairing in progress"));
}
void irStartPair(int dev, const String &amp;name) {
  if (irFreeSlot(dev, name.c_str()) &lt; 0) {
    Serial.println(F("[IR] That device already has 16 buttons - clear one first."));
    return;
  }
  irPairDev = dev;
  name.toCharArray(irPairName, sizeof(irPairName));
  irPairStart = millis();
  irPairing = true;
  irRecv.enableIRIn();
  Serial.println(F("--------------------------------"));
  Serial.print(F("[IR] Waiting for ")); Serial.print(irDevLabel[dev]);
  Serial.print(F(" \"")); Serial.print(irPairName); Serial.println(F("\" ..."));
  Serial.println(F("[IR] Point the remote at the receiver and press the button now (15 sec)."));
}
void irCaptureResult() {
  int dev = irPairDev;
  int slot = irFreeSlot(dev, irPairName);
  if (slot &lt; 0) {
    Serial.println(F("[IR] No free slot left for this device."));
  } else {
    IrButton &amp;b = irBtn[dev][slot];
    if (b.rawData) { free(b.rawData); b.rawData = nullptr; }   // free any previous code if this name/slot is
being re-paired
    strlcpy(b.name, irPairName, sizeof(b.name));
    b.used = true;
    // AC remotes often use long state-based codes that the simple protocol
    // sender can't replay, so ACs are always captured as raw timing; TV and
    // Projector use the compact protocol form when one is recognised.
    if (irResults.decode_type != UNKNOWN &amp;&amp; irResults.bits &gt; 0 &amp;&amp; irResults.bits &lt;= 64
&amp;&amp; dev != DEV_AC) {
      b.isRaw = false;
      b.protocol = irResults.decode_type;
      b.value = irResults.value;
      b.bits = irResults.bits;
      b.rawLen = 0;
      Serial.print(F("[IR] Captured ")); Serial.print(typeToString(irResults.decode_type));
      Serial.print(F(", ")); Serial.print(b.bits); Serial.println(F(" bits."));
    } else {
      uint16_t len = irResults.rawlen &gt; 1 ? irResults.rawlen - 1 : 0;
      if (len &gt; IR_MAX_RAW) len = IR_MAX_RAW;
      b.isRaw = true;
      b.protocol = UNKNOWN;
      b.rawLen = len;
      b.rawData = (uint16_t*)malloc((size_t)len * 2);
      if (!b.rawData) {
        Serial.println(F("[IR] Out of memory capturing this code."));
        b.used = false; b.rawLen = 0;
        irPairing = false; irRecv.disableIRIn();
        return;
      }
      for (uint16_t i = 0; i &lt; len; i++) b.rawData[i] = irResults.rawbuf[i + 1] * kRawTick;
      Serial.print(F("[IR] Captured as raw timing (")); Serial.print(len); Serial.println(F(" entries)."));
    }
    irSaveDevice(dev);
    Serial.print(F("[IR] Saved \"")); Serial.print(b.name); Serial.println(F("\". Ready for the next one, or
'list' to review."));
    logEvent(K_SYS, (String(irDevLabel[dev]) + " IR button paired: " + b.name).c_str());
  }
```

```
  irPairing = false;
  irRecv.disableIRIn();
}
bool irSendButton(int dev, const char* name) {
  int i = irFindButton(dev, name);
  if (i &lt; 0) return false;
  IrButton &amp;b = irBtn[dev][i];
  bool wasPairing = irPairing;
  irRecv.disableIRIn();
  if (b.isRaw) irSend.sendRaw(b.rawData, b.rawLen, 38);
  else irSend.send(b.protocol, b.value, b.bits);
  delay(30);
  if (wasPairing) irRecv.enableIRIn();
  return true;
}
void irHandleLine(String line) {
  line.trim();
  if (line.length() == 0) return;
  int sp = line.indexOf(' ');
  String cmd = sp &lt; 0 ? line : line.substring(0, sp);
  String rest = sp &lt; 0 ? "" : line.substring(sp + 1);
  rest.trim();
  cmd.toLowerCase();
  if (cmd == "help" || cmd == "?") { irPrintHelp(); return; }
  if (cmd == "cancel") {
    if (irPairing) { irPairing = false; irRecv.disableIRIn(); Serial.println(F("[IR] Pairing cancelled.")); }
    return;
  }
  if (cmd == "pair") {
    int sp2 = rest.indexOf(' ');
    if (sp2 &lt; 0) { Serial.println(F("[IR] Usage: pair ac|tv|proj &lt;name&gt;")); return; }
    int dev = irDevIndex(rest.substring(0, sp2));
    String name = rest.substring(sp2 + 1); name.trim();
    if (dev &lt; 0) { Serial.println(F("[IR] Unknown device - use ac, tv or proj.")); return; }
    if (name.length() == 0 || name.length() &gt; 19) { Serial.println(F("[IR] Give the button a short name (1-
19 chars), e.g. pair ac power")); return; }
    irStartPair(dev, name);
    return;
  }
  if (cmd == "list") {
    int dev = irDevIndex(rest);
    if (dev &lt; 0) { Serial.println(F("[IR] Usage: list ac|tv|proj")); return; }
    irPrintList(dev);
    return;
  }
  if (cmd == "send") {
    int sp2 = rest.indexOf(' ');
    if (sp2 &lt; 0) { Serial.println(F("[IR] Usage: send ac|tv|proj &lt;name&gt;")); return; }
    int dev = irDevIndex(rest.substring(0, sp2));
    String name = rest.substring(sp2 + 1); name.trim();
    if (dev &lt; 0) { Serial.println(F("[IR] Unknown device.")); return; }
    Serial.println(irSendButton(dev, name.c_str()) ? F("[IR] Sent.") : F("[IR] No button with that name."));
    return;
  }
  if (cmd == "clear") {
    int sp2 = rest.indexOf(' ');
    String devS = sp2 &lt; 0 ? rest : rest.substring(0, sp2);
    int dev = irDevIndex(devS);
    if (dev &lt; 0) { Serial.println(F("[IR] Usage: clear ac|tv|proj &lt;name|all&gt;")); return; }
    String name = sp2 &lt; 0 ? "" : rest.substring(sp2 + 1); name.trim();
    if (name.length() == 0 || name == "all") { irClearDevice(dev); Serial.println(F("[IR] Device cleared."));
}
    else { irDeleteButton(dev, name.c_str()); Serial.println(F("[IR] Removed (if it existed).")); }
    return;
  }
  Serial.println(F("[IR] Unknown command. Type 'help'."));
}
void irTick() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\r') continue;
    if (c == '\n') { irHandleLine(irLine); irLine = ""; }
    else if (irLine.length() &lt; 60) irLine += c;
  }
  if (irPairing) {
    if (irRecv.decode(&amp;irResults)) {
      irCaptureResult();
    } else if (millis() - irPairStart &gt; IR_PAIR_TIMEOUT) {
      Serial.println(F("[IR] Timed out - no signal seen. Type the pair command again."));
      irPairing = false;
      irRecv.disableIRIn();
    }
  }
}
// ---- HTTP: dashboard data for AC / TV / Projector ----
void handleIrList() {
  if (!checkAuth()) return;
  int dev = irDevIndex(server.arg("dev"));
  if (dev &lt; 0) { sendJson(400, "{\"err\":\"bad\"}"); return; }
  String j = "{\"dev\":\"" + String(irDevKey[dev]) + "\",\"btn\":[";
```

```
  bool first = true;
  for (int i = 0; i &lt; IR_MAX_BTN; i++) {
    if (!irBtn[dev][i].used) continue;
    if (!first) j += ","; first = false;
    j += "\"" + String(irBtn[dev][i].name) + "\"";
  }
  j += "]}";
  sendJson(200, j);
}
void handleIrSend() {
  if (!checkAuth()) return;
  int dev = irDevIndex(server.arg("dev"));
  String name = server.arg("name");
  if (dev &lt; 0 || name.length() == 0) { sendJson(400, "{\"err\":\"bad\"}"); return; }
  bool ok = irSendButton(dev, name.c_str());
  if (ok) logEvent(K_SYS, (String(irDevLabel[dev]) + " IR sent: " + name).c_str());
  // AC temperature display only: a numeric button name (e.g. "24") sets the
  // shown temp to that number; "temp_up"/"temp_down" nudge it by 1.
  if (ok &amp;&amp; dev == DEV_AC) {
    bool numeric = name.length() &gt; 0;
    for (unsigned int k = 0; k &lt; name.length(); k++) if (!isDigit(name[k])) { numeric = false; break; }
    if (numeric) {
      int t = name.toInt();
      if (t &gt;= AC_TEMP_MIN &amp;&amp; t &lt;= AC_TEMP_MAX &amp;&amp; t != acTemp) { acTemp = t; saveAcTemp(
); }
    } else if (name.equalsIgnoreCase("temp_up") &amp;&amp; acTemp &lt; AC_TEMP_MAX) { acTemp++; saveAcTemp();
}
    else if (name.equalsIgnoreCase("temp_down") &amp;&amp; acTemp &gt; AC_TEMP_MIN) { acTemp--; saveAcTemp();
}
  }
  String resp;
  if (ok) resp = (dev == DEV_AC) ? ("{\"ok\":true,\"temp\":" + String(acTemp) + "}") : "{\"ok\":true}";
  else resp = "{\"err\":\"missing\"}";
  sendJson(ok ? 200 : 404, resp);
}
void handleAcTemp() {
  if (!checkAuth()) return;
  sendJson(200, "{\"temp\":" + String(acTemp) + "}");
}
const char IR_PAGE[] PROGMEM = R"rawliteral(&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;&lt;head&gt;&lt;meta charset="utf-8"&gt;
&lt;meta name="viewport" content="width=device-width,initial-scale=1"&gt;
&lt;title&gt;IR Remotes - Smart Life&lt;/title&gt;
&lt;style&gt;
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:system-ui,Arial,sans-serif;background:#040a1c;color:#e8f1ff;padding:18px}
h1{font-size:20px;margin-bottom:4px}
a{color:#4cc3ff}
.tabs{display:flex;gap:8px;margin:16px 0}
.tabs button{flex:1;padding:10px;border-radius:10px;background:rgba(16,36,84,.6);border:1px solid rgba(96,165,
255,.25);color:#e8f1ff;font-size:15px}
.tabs button.on{background:#2b7fff}
.grid{display:grid;grid-template-columns:repeat(2,1fr);gap:10px}
.grid button{padding:16px 8px;border-radius:12px;background:rgba(16,36,84,.6);border:1px solid rgba(96,165,
255,.25);color:#e8f1ff;font-size:15px}
.grid button:active{background:#2b7fff}
.pwr{display:flex;gap:10px;margin-bottom:10px}
.pwr button{flex:1;padding:14px 8px;border-radius:12px;border:1px solid rgba(96,165,255,.25);font-size:15px;
font-weight:600;color:#e8f1ff}
.pwr button.pon{background:#1c7a3c}
.pwr button.poff{background:#7a2020}
.pwr button:active{filter:brightness(1.3)}
.actemp{display:none;align-items:center;gap:16px;justify-content:center;margin-bottom:14px;padding:14px;
border-radius:12px;background:rgba(16,36,84,.6);border:1px solid rgba(96,165,255,.25)}
.actemp button{width:40px;height:40px;border-radius:50%;background:#2b7fff;color:#fff;font-size:20px;border:
none}
.actemp span{font-size:22px;min-width:50px;text-align:center;display:inline-block}
.empty{color:#8ea8d8;padding:20px 0}
#msg{margin-top:14px;color:#8ea8d8;font-size:13px}
&lt;/style&gt;&lt;/head&gt;&lt;body&gt;
&lt;h1&gt;IR Remotes&lt;/h1&gt;
&lt;p&gt;&lt;a href="/"&gt;&amp;larr; back to dashboard&lt;/a&gt;&lt;/p&gt;
&lt;div class="tabs"&gt;
 &lt;button data-d="ac" class="on"&gt;AC&lt;/button&gt;
 &lt;button data-d="tv"&gt;TV&lt;/button&gt;
 &lt;button data-d="proj"&gt;Projector&lt;/button&gt;
&lt;/div&gt;
&lt;div id="pwr" class="pwr"&gt;&lt;/div&gt;
&lt;div id="actemp" class="actemp"&gt;
 &lt;button id="tdown"&gt;-&lt;/button&gt;
 &lt;span id="tval"&gt;--&lt;/span&gt;&amp;deg;C
 &lt;button id="tup"&gt;+&lt;/button&gt;
&lt;/div&gt;
&lt;div id="grid" class="grid"&gt;&lt;/div&gt;
&lt;p id="msg"&gt;&lt;/p&gt;
&lt;script&gt;
let dev='ac';
async function api(u){const r=await fetch(u,{credentials:'same-origin'});if(r.status===401){location.href='/';
throw 0}return r}
async function sendBtn(name,label){
```

```
 document.getElementById('msg').textContent='Sending '+(label||name)+' ...';
 try{
  const r=await api('/api/irsend?dev='+dev+'&amp;name='+encodeURIComponent(name));const jr=await r.json();
  document.getElementById('msg').textContent=jr.ok?('Sent: '+(label||name)):'Failed to send';
  if(jr.ok&amp;&amp;dev==='ac'&amp;&amp;jr.temp!==undefined)document.getElementById('tval').textContent=jr.
temp;
 }catch(e){document.getElementById('msg').textContent='Board not reachable'}
}
async function loadTemp(){
 try{const r=await api('/api/actemp');const j=await r.json();document.getElementById('tval').textContent=j.
temp}
 catch(e){}
}
async function load(){
 document.querySelectorAll('.tabs button').forEach(b=&gt;b.classList.toggle('on',b.dataset.d===dev));
 const g=document.getElementById('grid');g.innerHTML='';
 const pwr=document.getElementById('pwr');pwr.innerHTML='';
 document.getElementById('actemp').style.display=(dev==='ac')?'flex':'none';
 if(dev==='ac')loadTemp();
 try{
  const r=await api('/api/irlist?dev='+dev);const j=await r.json();
  const rest=j.btn.filter(name=&gt;{
   const low=name.toLowerCase();
   if(low==='on'||low==='off'){
    const b=document.createElement('button');
    b.textContent=low==='on'?'Power ON':'Power OFF';
    b.className=low==='on'?'pon':'poff';
    b.onclick=()=&gt;sendBtn(name,b.textContent);
    pwr.appendChild(b);
    return false;
   }
   return true;
  });
  if(!rest.length){
   if(!pwr.children.length)g.innerHTML='&lt;p class="empty"&gt;No buttons paired yet for this device.&lt;
br&gt;In the Serial Monitor type: pair '+dev+' &amp;lt;name&amp;gt;&lt;/p&gt;';
   return;
  }
  rest.forEach(name=&gt;{
   const b=document.createElement('button');b.textContent=name;
   b.onclick=()=&gt;sendBtn(name);
   g.appendChild(b);
  });
 }catch(e){}
}
document.querySelectorAll('.tabs button').forEach(b=&gt;b.onclick=()=&gt;{dev=b.dataset.d;load()});
document.getElementById('tup').onclick=()=&gt;sendBtn('temp_up','Temp +1');
document.getElementById('tdown').onclick=()=&gt;sendBtn('temp_down','Temp -1');
load();
&lt;/script&gt;
&lt;/body&gt;&lt;/html&gt;)rawliteral";
void handleIrPage() {
  server.sendHeader("Cache-Control", "no-cache");
  server.send_P(200, "text/html", IR_PAGE);
}
// ============================================================================
//  Web handlers
// ============================================================================
void sendJson(int code, const String &amp;s) {
  server.sendHeader("Cache-Control", "no-store");
  server.send(code, "application/json", s);
}
bool checkAuth() {
  if (authed()) return true;
  sendJson(401, "{\"err\":\"auth\"}");
  return false;
}
int idxOf(int id) {
  for (int i = 0; i &lt; 3; i++) if (relayId[i] == id) return i;
  return -1;
}
int parseHM(const String &amp;s) {
  int c = s.indexOf(':');
  if (c &lt; 1) return -1;
  int h = s.substring(0, c).toInt();
  int m = s.substring(c + 1).toInt();
  if (h &lt; 0 || h &gt; 23 || m &lt; 0 || m &gt; 59) return -1;
  return h * 60 + m;
}
String hhmm(int m) {
  char b[8];
  snprintf(b, sizeof(b), "%02d:%02d", m / 60, m % 60);
  return String(b);
}
String tf(bool v) { return v ? "true" : "false"; }
void handleLogin() {
  String ip = server.client().remoteIP().toString();
  char b[48];
  if (server.arg("u") == LOGIN_USER &amp;&amp; server.arg("p") == loginPass) {
    snprintf(b, sizeof(b), "Signed in from %s", ip.c_str());
```

```
    logEvent(K_LOGIN, b);
    String t = newToken();
    sess[sessNext] = t;
    sessNext = (sessNext + 1) % MAX_SESS;
    server.sendHeader("Set-Cookie", "SLTOK=" + t + "; Path=/; SameSite=Strict");
    sendJson(200, "{\"ok\":true}");
  } else {
    snprintf(b, sizeof(b), "Wrong password from %s", ip.c_str());
    logEvent(K_LOGIN, b);
    sendJson(401, "{\"ok\":false}");
  }
}
void handleLogout() {
  String t = cookieToken();
  for (int k = 0; k &lt; MAX_SESS; k++) if (t.length() == 32 &amp;&amp; sess[k] == t) sess[k] = "";
  server.sendHeader("Set-Cookie", "SLTOK=; Path=/; Max-Age=0");
  sendJson(200, "{\"ok\":true}");
}
void handleState() {
  if (!checkAuth()) return;
  struct tm t;
  bool ok = nowTime(t);
  char tb[12] = "--:--:--";
  if (ok) strftime(tb, sizeof(tb), "%H:%M:%S", &amp;t);
  String j;
  j.reserve(900);
  j += "{\"time\":\""; j += tb; j += "\",\"synced\":"; j += tf(ok);
  j += ",\"ip\":\"" + WiFi.localIP().toString() + "\",\"rssi\":" + String(WiFi.RSSI());
  j += ",\"r\":[";
  for (int i = 0; i &lt; 3; i++) {
    if (i) j += ",";
    j += "{\"id\":" + String(relayId[i]);
    j += ",\"on\":" + tf(relayEffective(i));
    j += ",\"auto\":" + tf(relayOwned(i));
    j += ",\"se\":" + tf(sched[i].en);
    j += ",\"on_t\":\"" + hhmm(sched[i].onMin) + "\"";
    j += ",\"off_t\":\"" + hhmm(sched[i].offMin) + "\"}";
  }
  j += "],\"ldr\":{\"en\":" + tf(ldrEn) + ",\"det\":" + tf(ldrEn &amp;&amp; ldrState == 1) + ",\"raw\":" +
String(ldrRaw) + "}";
  j += ",\"pir\":{\"en\":" + tf(pirEn) + ",\"det\":" + tf(pirEn &amp;&amp; holding) + "}";
  j += ",\"gas\":{\"en\":" + tf(gasEn) + ",\"det\":" + tf(gasEn &amp;&amp; gasState == 1) + ",\"raw\":" +
String(gasRaw) + "},\"ev\":" + String(evTotal) + "}";
  sendJson(200, j);
}
void handleRelay() {
  if (!checkAuth()) return;
  int i = idxOf(server.arg("id").toInt());
  if (i &lt; 0) { sendJson(400, "{\"err\":\"bad\"}"); return; }
  if (relayOwned(i)) { sendJson(409, "{\"err\":\"auto\"}"); return; }
  setManual(i, server.arg("s") == "1", "button");
  applyRelays();
  sendJson(200, "{\"ok\":true}");
}
void handleSched() {
  if (!checkAuth()) return;
  int i = idxOf(server.arg("id").toInt());
  int on = parseHM(server.arg("on"));
  int off = parseHM(server.arg("off"));
  if (i &lt; 0 || on &lt; 0 || off &lt; 0) { sendJson(400, "{\"err\":\"bad\"}"); return; }
  sched[i].en = (server.arg("en") == "1");
  sched[i].onMin = on;
  sched[i].offMin = off;
  saveSched(i);
  lastMinute = -1;   // re-check the current minute right away
  sendJson(200, "{\"ok\":true}");
}
void handleSensor() {
  if (!checkAuth()) return;
  String n = server.arg("n");
  bool s = (server.arg("s") == "1");
  if (n == "ldr") {
    if (ldrEn != s) logEvent(K_SENSOR, s ? "LDR sensor ON" : "LDR sensor OFF");
    ldrEn = s; ldrState = 0; prevLdr = 0; lightCandidate = false;
  } else if (n == "pir") {
    if (pirEn != s) logEvent(K_SENSOR, s ? "PIR sensor ON" : "PIR sensor OFF");
    pirEn = s; holding = false; prevPir = false;
  } else if (n == "gas") {
    if (gasEn != s) logEvent(K_SENSOR, s ? "Gas sensor ON" : "Gas sensor OFF");
    gasEn = s; gasState = 0; prevGas = 0; clearCandidate = false;
  } else { sendJson(400, "{\"err\":\"bad\"}"); return; }
  applyRelays();
  applyOutputs();
  sendJson(200, "{\"ok\":true}");
}
void handleLog() {
  if (!checkAuth()) return;
  String j;
  j.reserve(4600);
  j += "{\"e\":[";
```

```
  for (int n = 0; n &lt; evCount; n++) {
    int idx = (evHead - 1 - n + LOG_MAX * 2) % LOG_MAX;     // newest first
    if (n) j += ",";
    j += "{\"t\":\"" + evTime(idx) + "\",\"k\":" + String(evs[idx].k) + ",\"m\":\"" + String(evs[idx].m) + "\
"}";
  }
  j += "]}";
  sendJson(200, j);
}
void handleLogClear() {
  if (!checkAuth()) return;
  evHead = 0;
  evCount = 0;
  logEvent(K_SYS, "Log cleared");
  sendJson(200, "{\"ok\":true}");
}
String maskTok(const String &amp;t) {
  if (t.length() == 0) return "";
  if (t.length() &lt; 8) return "set";
  return "..." + t.substring(t.length() - 4);
}
void handleTgGet() {
  if (!checkAuth()) return;
  xSemaphoreTake(cfgMux, portMAX_DELAY);
  String ids = tgIds;
  String tok = maskTok(tgToken);
  xSemaphoreGive(cfgMux);
  uint32_t at = tgAtMs;
  String j = "{\"on\":" + tf(tgOn) + ",\"tok\":\"" + tok + "\",\"ids\":\"" + ids + "\",\"code\":" + String((
int)tgCode) +
             ",\"ago\":" + String(at ? (unsigned long)((millis() - at) / 1000) : 0UL) + "}";
  sendJson(200, j);
}
void handleTgSet() {
  if (!checkAuth()) return;
  String tok = server.arg("tok");
  String ids = server.arg("ids");
  tok.trim();
  tok.replace(" ", "");
  ids.trim();
  ids.replace("\"", "");
  ids.replace("\\", "");
  if (tok.length() &gt; 60 || ids.length() &gt; 95) { sendJson(400, "{\"err\":\"long\"}"); return; }
  xSemaphoreTake(cfgMux, portMAX_DELAY);
  if (tok.length()) tgToken = tok;      // empty = keep the saved token
  tgIds = ids;
  tgOn = (server.arg("on") == "1");
  prefs.begin("smartlife", false);
  prefs.putString("tgt", tgToken);
  prefs.putString("tgi", tgIds);
  prefs.putBool("tgo", tgOn);
  prefs.end();
  xSemaphoreGive(cfgMux);
  logEvent(K_TG, tgOn ? "Telegram settings saved (alerts ON)" : "Telegram settings saved (alerts OFF)");
  sendJson(200, "{\"ok\":true}");
}
void handleTgTest() {
  if (!checkAuth()) return;
  tgQueue("■ Smart Life test message. Telegram alerts are working.");
  sendJson(200, "{\"ok\":true}");
}
void handlePasswd() {
  if (!checkAuth()) return;
  String cur = server.arg("cur");
  String nw = server.arg("new");
  if (cur != loginPass) { sendJson(403, "{\"err\":\"wrong\"}"); return; }
  if (nw.length() &lt; 4 || nw.length() &gt; 32) { sendJson(400, "{\"err\":\"len\"}"); return; }
  loginPass = nw;
  prefs.begin("smartlife", false);
  prefs.putString("pw", loginPass);
  prefs.end();
  String me = cookieToken();                        // sign out every other device
  for (int k = 0; k &lt; MAX_SESS; k++) if (sess[k] != me) sess[k] = "";
  logEvent(K_LOGIN, "Password changed");
  sendJson(200, "{\"ok\":true}");
}
void handleNotFound() {
  server.send(404, "text/plain", "Not found");
}
// ====================================================================
//  The web page (login + voice home + relays + LDR + PIR + gas)
// ====================================================================
const char PAGE[] PROGMEM = R"rawliteral(&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;&lt;head&gt;
&lt;meta charset="utf-8"&gt;
&lt;meta name="viewport" content="width=device-width,initial-scale=1"&gt;
&lt;meta name="theme-color" content="#050d26"&gt;
&lt;title&gt;Smart Life with ESP32&lt;/title&gt;
&lt;style&gt;
*{box-sizing:border-box;margin:0;padding:0;-webkit-tap-highlight-color:transparent}
```

```
[hidden]{display:none!important}
:root{--bg:#040a1c;--panel:rgba(16,36,84,.55);--line:rgba(96,165,255,.22);--ink:#e8f1ff;--mute:#8ea8d8;--blue:
#2b7fff;--sky:#4cc3ff;--glow:rgba(43,127,255,.45);--red:#ff5468}
html,body{min-height:100%}
body{font-family:"Segoe UI",system-ui,-apple-system,Roboto,"Helvetica Neue",Arial,sans-serif;color:var(--ink);
background:var(--bg);overflow-x:hidden}
body:before{content:"";position:fixed;inset:0;z-index:-1;background:radial-gradient(900px 520px at 12% -8%,
rgba(43,127,255,.32),transparent 60%),radial-gradient(800px 480px at 105% 8%,rgba(76,195,255,.18),transparent
60%),linear-gradient(180deg,#050d26,#030817)}
button{font:inherit;color:inherit;border:0;cursor:pointer;background:none}
input{font:inherit;color:inherit}
button:focus-visible,input:focus-visible{outline:2px solid var(--sky);outline-offset:2px}
/* logo */
.logo{width:92px;height:92px;margin:0 auto 18px;border-radius:26px;display:grid;place-items:center;background:
linear-gradient(145deg,#2b7fff,#4cc3ff);animation:ring 1.6s ease-out infinite}
.logo svg{width:52px;height:52px;fill:none;stroke:#04122e;stroke-width:1.8;stroke-linecap:round;stroke-
linejoin:round}
@keyframes ring{0%{box-shadow:0 0 0 0 rgba(76,195,255,.6)}100%{box-shadow:0 0 0 34px rgba(76,195,255,0)}}
/* splash */
#splash{position:fixed;inset:0;z-index:50;display:grid;place-items:center;text-align:center;padding:24px;
background:radial-gradient(700px 420px at 50% 40%,#0f3a99 0%,#050d26 70%);transition:opacity .45s}
#splash.out{opacity:0;pointer-events:none}
#splash h1{font-size:46px;font-weight:800;letter-spacing:-.03em}
#splash h2{font-size:18px;font-weight:500;color:var(--sky);margin-top:2px}
#splash p{max-width:340px;margin:14px auto 0;color:var(--mute);line-height:1.5}
.bar{width:180px;height:4px;border-radius:4px;background:rgba(255,255,255,.1);margin:28px auto 0;overflow:
hidden}
.bar i{display:block;height:100%;width:0;background:linear-gradient(90deg,var(--blue),var(--sky));animation:
fill 2s linear forwards}
@keyframes fill{to{width:100%}}
/* login */
#login{min-height:100vh;display:grid;place-items:center;padding:20px}
.lcard{width:100%;max-width:410px;padding:30px 26px;border-radius:24px;background:var(--panel);border:1px
solid var(--line);backdrop-filter:blur(18px);-webkit-backdrop-filter:blur(18px);box-shadow:0 30px 80px rgba(0,
20,80,.55)}
.lcard .logo{width:64px;height:64px;border-radius:19px;animation:none;margin:0 0 16px;box-shadow:0 8px 26px
var(--glow)}
.lcard .logo svg{width:36px;height:36px}
.lcard h1{font-size:32px;font-weight:800;letter-spacing:-.03em}
.lcard .sub{color:var(--sky);font-weight:500;margin-top:2px}
.lcard .desc{color:var(--mute);line-height:1.55;margin:12px 0 22px;font-size:14.5px}
.fld{display:block;margin-bottom:14px;font-size:13.5px;color:var(--mute);position:relative}
.fld input{display:block;width:100%;margin-top:6px;padding:13px 14px;border-radius:12px;border:1px solid var(-
-line);background:rgba(4,12,40,.6);font-size:16px;outline:0}
.fld input:focus{border-color:var(--sky);box-shadow:0 0 0 3px rgba(76,195,255,.18)}
.fld .eye{position:absolute;right:8px;bottom:8px;padding:6px 10px;border-radius:8px;background:rgba(76,195,
255,.12);color:var(--sky);font-size:13px}
.cta{width:100%;padding:14px;margin-top:6px;border-radius:12px;font-weight:700;font-size:16px;background:
linear-gradient(135deg,var(--blue),var(--sky));color:#03102b;box-shadow:0 8px 26px var(--glow)}
.cta:disabled{opacity:.6}
.err{min-height:20px;margin-top:12px;color:#ff8b99;font-size:14px}
/* app shell */
#app{max-width:1040px;margin:0 auto;padding:18px 16px 40px}
header{display:flex;align-items:center;justify-content:space-between;gap:12px}
.brand{display:flex;align-items:center;gap:12px}
.brand .logo{width:42px;height:42px;flex:none;border-radius:13px;animation:none;margin:0;box-shadow:0 6px
20px var(--glow)}
.brand .logo svg{width:28px;height:28px}
.brand h1{font-size:23px;font-weight:800;letter-spacing:-.03em;line-height:1.1;white-space:nowrap}
.brand small{display:block;color:var(--sky);font-size:13.5px;font-weight:500;margin-top:2px}
.hr{display:flex;align-items:center;gap:8px}
#clock{font-variant-numeric:tabular-nums;padding:8px 10px;border-radius:12px;background:var(--panel);border:
1px solid var(--line);font-size:13px;white-space:nowrap}
.ghost{padding:8px 12px;border-radius:12px;border:1px solid var(--line);color:var(--mute);font-size:13px;
white-space:nowrap}
.live{position:relative;margin-top:14px;height:56px;border-radius:14px;overflow:hidden;border:1px solid var(--
line);background:linear-gradient(180deg,rgba(43,127,255,.12),rgba(4,12,40,.35))}
.live canvas{position:absolute;inset:0;width:100%;height:100%}
.live .tag{position:absolute;left:12px;top:8px;display:flex;align-items:center;gap:7px;font-size:12.5px;color:
var(--sky)}
.live .dot{width:8px;height:8px;border-radius:50%;background:var(--sky);box-shadow:0 0 10px var(--sky)}
.live.off .dot{background:var(--red);box-shadow:0 0 10px var(--red)}
.live.off .tag{color:#ff8b99}
nav{display:flex;gap:8px;margin:16px 0;overflow-x:auto;padding-bottom:2px;scrollbar-width:none}
nav::-webkit-scrollbar{display:none}
nav button{position:relative;flex:1 1 0;min-width:0;padding:11px 6px;border-radius:14px;background:var(--
panel);border:1px solid var(--line);color:var(--mute);font-weight:600}
nav button.sel{color:#03102b;background:linear-gradient(135deg,var(--blue),var(--sky));border-color:
transparent;box-shadow:0 6px 22px var(--glow)}
nav .td{position:absolute;top:6px;right:8px;width:8px;height:8px;border-radius:50%;background:var(--red);box-
shadow:0 0 8px var(--red)}
.panel{display:none}
.panel.sel{display:block}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(270px,1fr));gap:14px}
.one{max-width:520px;margin:0 auto}
/* cards */
.card{padding:18px;border-radius:20px;background:var(--panel);border:1px solid var(--line);backdrop-filter:
blur(14px);-webkit-backdrop-filter:blur(14px);transition:border-color .25s,box-shadow .25s}
.ch{display:flex;align-items:center;justify-content:space-between;margin-bottom:14px}
.ch h3{font-size:18px;font-weight:700}
```

```
.pill{padding:4px 12px;border-radius:99px;font-size:13px;font-weight:700;background:rgba(142,168,216,.16);
color:var(--mute)}
.irgrid{display:grid;grid-template-columns:repeat(2,1fr);gap:10px}
.irgrid button{padding:16px 8px;border-radius:16px;background:var(--panel);border:1px solid var(--line);color:
var(--ink);font-weight:600;font-size:15px}
.irgrid button:active{background:linear-gradient(135deg,var(--blue),var(--sky));color:#03102b}
.irempty{color:var(--mute);padding:10px 2px;line-height:1.5}
.irempty code{background:rgba(142,168,216,.16);padding:1px 6px;border-radius:6px}
.pill.lit{background:rgba(76,195,255,.18);color:var(--sky)}
.pair{display:grid;grid-template-columns:1fr 1fr;gap:10px}
.b{padding:13px;border-radius:12px;font-weight:700;background:rgba(142,168,216,.12);border:1px solid var(--
line);color:var(--mute);transition:transform .12s}
.b:active{transform:scale(.97)}
.b.on.act{background:linear-gradient(135deg,var(--blue),var(--sky));color:#03102b;border-color:transparent;
box-shadow:0 6px 20px var(--glow)}
.b.off.act{background:rgba(142,168,216,.28);color:var(--ink)}
.note{font-size:13px;color:#ffd58a;margin-bottom:12px;line-height:1.45}
.info{font-size:13px;color:var(--mute);margin-top:12px;line-height:1.45}
.card.auto .pair,.card.auto .timer{opacity:.45}
/* timer */
.timer{margin-top:16px;padding-top:14px;border-top:1px solid var(--line)}
.trow{display:flex;align-items:center;justify-content:space-between;font-weight:600}
.sw{position:relative;width:46px;height:26px}
.sw input{position:absolute;inset:0;width:100%;height:100%;opacity:0;cursor:pointer;z-index:1}
.sw i{position:absolute;inset:0;border-radius:99px;background:rgba(142,168,216,.25);transition:.2s}
.sw i:after{content:"";position:absolute;left:3px;top:3px;width:20px;height:20px;border-radius:50%;background:
#fff;transition:.2s}
.sw input:checked+i{background:var(--blue)}
.sw input:checked+i:after{transform:translateX(20px)}
.tin{display:grid;grid-template-columns:1fr 1fr;gap:10px;margin:12px 0}
.tin label{font-size:13px;color:var(--mute)}
.tin input{display:block;width:100%;margin-top:5px;padding:10px;border-radius:10px;border:1px solid var(--
line);background:rgba(4,12,40,.6);color-scheme:dark;font-size:16px}
.save{width:100%;background:rgba(76,195,255,.14);color:var(--sky);border-color:transparent}
/* sensors */
.read{display:flex;align-items:center;gap:16px;margin:4px 0 14px}
.ico{width:64px;height:64px;flex:none;border-radius:20px;display:grid;place-items:center;background:rgba(142,
168,216,.12);color:var(--mute);transition:.25s}
.ico svg{width:32px;height:32px;fill:none;stroke:currentColor;stroke-width:2;stroke-linecap:round;stroke-
linejoin:round}
.big{font-size:26px;font-weight:800;letter-spacing:-.02em}
.sub{color:var(--mute);margin-top:2px}
.raw{font-size:13px;color:var(--mute);margin:-4px 0 14px;font-variant-numeric:tabular-nums}
.card.det{border-color:var(--sky);box-shadow:0 0 34px rgba(76,195,255,.25)}
.card.det .ico{background:linear-gradient(135deg,var(--blue),var(--sky));color:#03102b}
.card.danger{border-color:var(--red);animation:warn 1s ease-in-out infinite alternate}
.card.danger .ico{background:var(--red);color:#fff}
@keyframes warn{from{box-shadow:0 0 22px rgba(255,84,104,.25)}to{box-shadow:0 0 56px rgba(255,84,104,.55)}}
/* voice home */
.voice{text-align:center;padding:30px 18px}
.mic{position:relative;width:132px;height:132px;border-radius:50%;margin:6px auto 20px;background:linear-
gradient(145deg,var(--blue),var(--sky));display:grid;place-items:center;box-shadow:0 12px 40px var(--glow)}
.mic svg{width:52px;height:52px;fill:none;stroke:#03102b;stroke-width:2;stroke-linecap:round;stroke-linejoin:
round}
.mic.rec:before,.mic.rec:after{content:"";position:absolute;inset:0;border-radius:50%;border:2px solid var(--
sky);animation:rip 1.4s ease-out infinite}
.mic.rec:after{animation-delay:.7s}
@keyframes rip{from{transform:scale(1);opacity:.8}to{transform:scale(1.7);opacity:0}}
#state{font-size:21px;font-weight:700}
#heard{color:var(--mute);min-height:24px;margin-top:8px}
#said{margin-top:6px;min-height:24px;color:var(--sky);font-weight:600}
.chips{display:flex;flex-wrap:wrap;gap:8px;justify-content:center;margin:20px 0 4px}
.chips button{padding:9px 14px;border-radius:99px;background:rgba(76,195,255,.10);border:1px solid var(--
line);font-size:14px}
.say{display:flex;gap:8px;max-width:460px;margin:18px auto 0}
.say input{flex:1;min-width:0;padding:12px 14px;border-radius:12px;border:1px solid var(--line);background:
rgba(4,12,40,.6);outline:0;font-size:16px}
.say button{padding:12px 18px;border-radius:12px;font-weight:700;background:linear-gradient(135deg,var(--
blue),var(--sky));color:#03102b}
footer{margin-top:22px;display:flex;flex-wrap:wrap;gap:8px;justify-content:center;font-size:13px;color:var(--
mute)}
footer span{padding:6px 12px;border-radius:99px;background:var(--panel);border:1px solid var(--line)}
#toast{position:fixed;left:50%;bottom:24px;transform:translate(-50%,30px);opacity:0;padding:12px 18px;border-
radius:14px;background:#0d2a6b;border:1px solid var(--line);box-shadow:0 10px 30px rgba(0,0,0,.4);transition:.
25s;z-index:60;max-width:90vw;text-align:center;pointer-events:none}
#toast.show{opacity:1;transform:translate(-50%,0)}
@media (prefers-reduced-motion:reduce){*{animation:none!important;transition:none!important}.bar i{width:
100%}}
&lt;/style&gt;&lt;/head&gt;
&lt;body&gt;
&lt;svg width="0" height="0" style="position:absolute" aria-hidden="true"&gt;&lt;symbol id="lg" viewBox="0 0
24 24"&gt;&lt;path d="M3 11.5 12 4l9 7.5"/&gt;&lt;path d="M5.5 10v9.5h13V10"/&gt;&lt;path d="M8.6 13.6a4.9 4.
9 0 0 1 6.8 0"/&gt;&lt;path d="M10.4 15.7a2.3 2.3 0 0 1 3.2 0"/&gt;&lt;circle cx="12" cy="17.7" r=".7"
fill="#04122e"/&gt;&lt;/symbol&gt;&lt;/svg&gt;
&lt;div id="splash"&gt;
  &lt;div&gt;
    &lt;div class="logo"&gt;&lt;svg&gt;&lt;use href="#lg"/&gt;&lt;/svg&gt;&lt;/div&gt;
    &lt;h1&gt;Smart Life&lt;/h1&gt;
    &lt;h2&gt;with ESP32&lt;/h2&gt;
    &lt;p&gt;Control your home with your voice, timers and live safety sensors, all running on one board.&lt;/
```

```
p&gt;
    &lt;div class="bar"&gt;&lt;i&gt;&lt;/i&gt;&lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;
&lt;main id="login" hidden&gt;
  &lt;form class="lcard" id="lf" autocomplete="on"&gt;
    &lt;div class="logo"&gt;&lt;svg&gt;&lt;use href="#lg"/&gt;&lt;/svg&gt;&lt;/div&gt;
    &lt;h1&gt;Smart Life&lt;/h1&gt;
    &lt;div class="sub"&gt;with ESP32&lt;/div&gt;
    &lt;p class="desc"&gt;Switch relays by voice or timer, and watch the light, motion and gas sensors live.
Sign in to open your home.&lt;/p&gt;
    &lt;label class="fld"&gt;Username&lt;input id="lu" type="text" autocomplete="username"
autocapitalize="none" required&gt;&lt;/label&gt;
    &lt;label class="fld"&gt;Password&lt;input id="lp" type="password" autocomplete="current-password"
required&gt;&lt;button class="eye" type="button" id="eye"&gt;Show&lt;/button&gt;&lt;/label&gt;
    &lt;button class="cta" id="lb" type="submit"&gt;Sign in&lt;/button&gt;
    &lt;p class="err" id="le" role="alert"&gt;&lt;/p&gt;
  &lt;/form&gt;
&lt;/main&gt;
&lt;div id="app" hidden&gt;
  &lt;header&gt;
    &lt;div class="brand"&gt;
      &lt;div class="logo"&gt;&lt;svg&gt;&lt;use href="#lg"/&gt;&lt;/svg&gt;&lt;/div&gt;
      &lt;div&gt;&lt;h1&gt;Smart Life&lt;/h1&gt;&lt;small&gt;with ESP32&lt;/small&gt;&lt;/div&gt;
    &lt;/div&gt;
    &lt;div class="hr"&gt;&lt;span id="clock"&gt;--:--:--&lt;/span&gt;&lt;button class="ghost" id="out"&gt;
Log out&lt;/button&gt;&lt;/div&gt;
  &lt;/header&gt;
  &lt;div class="live" id="livebox"&gt;&lt;canvas id="live"&gt;&lt;/canvas&gt;&lt;div class="tag"&gt;&lt;i
class="dot"&gt;&lt;/i&gt;&lt;span id="livetxt"&gt;Live&lt;/span&gt;&lt;/div&gt;&lt;/div&gt;
  &lt;nav id="tabs"&gt;
    &lt;button class="sel" data-tab="home"&gt;Home&lt;/button&gt;
    &lt;button data-tab="relays"&gt;Relays&lt;/button&gt;
    &lt;button data-tab="ldr"&gt;LDR&lt;i class="td" id="td-ldr" hidden&gt;&lt;/i&gt;&lt;/button&gt;
    &lt;button data-tab="pir"&gt;PIR&lt;i class="td" id="td-pir" hidden&gt;&lt;/i&gt;&lt;/button&gt;
    &lt;button data-tab="gas"&gt;Gas&lt;i class="td" id="td-gas" hidden&gt;&lt;/i&gt;&lt;/button&gt;
    &lt;button data-tab="ac"&gt;AC&lt;/button&gt;
    &lt;button data-tab="tv"&gt;TV&lt;/button&gt;
    &lt;button data-tab="proj"&gt;Projector&lt;/button&gt;
  &lt;/nav&gt;
  &lt;section class="panel sel" id="p-home"&gt;
    &lt;div class="card voice"&gt;
      &lt;button class="mic" id="mic" aria-label="Tap to speak"&gt;&lt;svg viewBox="0 0 24 24"&gt;&lt;rect
x="9" y="3" width="6" height="12" rx="3"/&gt;&lt;path d="M5 11a7 7 0 0 0 14 0M12 18v3"/&gt;&lt;/svg&gt;&lt;/
button&gt;
      &lt;div id="state"&gt;Tap the mic and speak&lt;/div&gt;
      &lt;div id="heard"&gt;&lt;/div&gt;
      &lt;div id="said"&gt;&lt;/div&gt;
      &lt;div class="chips"&gt;
        &lt;button data-cmd="Relay 1 on"&gt;Relay 1 on&lt;/button&gt;
        &lt;button data-cmd="Relay 1 off"&gt;Relay 1 off&lt;/button&gt;
        &lt;button data-cmd="Relay 3 on"&gt;Relay 3 on&lt;/button&gt;
        &lt;button data-cmd="Relay 4 off"&gt;Relay 4 off&lt;/button&gt;
        &lt;button data-cmd="All on"&gt;All on&lt;/button&gt;
        &lt;button data-cmd="All off"&gt;All off&lt;/button&gt;
      &lt;/div&gt;
      &lt;div class="say"&gt;&lt;input id="typed" type="text" placeholder="Or type: relay 1 on"
autocomplete="off"&gt;&lt;button id="send"&gt;Send&lt;/button&gt;&lt;/div&gt;
    &lt;/div&gt;
  &lt;/section&gt;
  &lt;section class="panel" id="p-relays"&gt;&lt;div class="grid" id="relays"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-ldr"&gt;&lt;div class="one" id="s-ldr"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-pir"&gt;&lt;div class="one" id="s-pir"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-gas"&gt;&lt;div class="one" id="s-gas"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-ac"&gt;&lt;div class="irgrid" id="ir-ac"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-tv"&gt;&lt;div class="irgrid" id="ir-tv"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;section class="panel" id="p-proj"&gt;&lt;div class="irgrid" id="ir-proj"&gt;&lt;/div&gt;&lt;/section&gt;
  &lt;footer&gt;&lt;span&gt;Board IP &lt;b id="ip"&gt;-&lt;/b&gt;&lt;/span&gt;&lt;span&gt;Wi-Fi &lt;b
id="rssi"&gt;-&lt;/b&gt;&lt;/span&gt;&lt;/footer&gt;
&lt;/div&gt;
&lt;div id="toast" role="status"&gt;&lt;/div&gt;
&lt;script&gt;
const $=id=&gt;document.getElementById(id);
const RL=[1,3,4];
const SENS=[
 {k:'ldr',name:'LDR sensor',yes:'Dark room',no:'Room is bright',info:'While the sensor is on, darkness
switches Relay 4 on. Light switches it off after 3 seconds.',
  ico:'&lt;circle cx="12" cy="12" r="4"/&gt;&lt;path d="M12 2v3M12 19v3M2 12h3M19 12h3M4.9 4.9l2.1 2.1M17
17l2.1 2.1M4.9 19.1 7 17M17 7l2.1-2.1"/&gt;'},
 {k:'pir',name:'PIR sensor',yes:'Motion sensed',no:'No motion',info:'While the sensor is on, motion switches
Relay 3 on for 2 seconds.',
  ico:'&lt;circle cx="12" cy="5" r="2.5"/&gt;&lt;path d="M12 8.5v5.5l-3 6M12 14l3 6M7 11.5l5-2.5 5 2.5"/&gt;
'},
 {k:'gas',name:'Gas sensor',yes:'Gas or smoke in the air',no:'Air is clear',info:'While the sensor is on, gas
sounds the buzzer and lights the LED. It stops after 3 seconds of clean air.',
  ico:'&lt;path d="M12 3c1 3.5 5 5 5.5 5 10a5 5 0 0 1-10 0c0-2 1-3.2 2-4.2 0 2 .8 3 2 3 0-3-1-5 1-8.8z"/&gt;'}
];
let online=true,appOn=false,timer=null,fails=0,dirty={};
const IRD=['ac','tv','proj'],irLoaded={};
/* ---------- IR (AC / TV / Projector) buttons ---------- */
```

```
async function loadIr(dev){
 const g=$('ir-'+dev);
 try{
  const r=await api('/api/irlist?dev='+dev);
  const list=(r.j&amp;&amp;r.j.btn)||[];
  if(!list.length){g.innerHTML='&lt;p class="irempty"&gt;No buttons paired yet for this device.&lt;br&gt;In
the Serial Monitor type &lt;code&gt;pair '+dev+' name&lt;/code&gt; (e.g. &lt;code&gt;pair '+dev+' power&lt;/
code&gt;), then press that button on the remote.&lt;/p&gt;';return}
  g.innerHTML=list.map(n=&gt;`&lt;button data-ir="${dev}" data-name="${n}"&gt;${n}&lt;/button&gt;`).join('');
  irLoaded[dev]=true;
 }catch(e){g.innerHTML='&lt;p class="irempty"&gt;Could not load buttons - board not reachable.&lt;/p&gt;'}
}
/* ---------- build cards ---------- */
$('relays').innerHTML=RL.map(id=&gt;`&lt;article class="card" id="r${id}"&gt;
&lt;div class="ch"&gt;&lt;h3&gt;Relay ${id}&lt;/h3&gt;&lt;span class="pill" id="rp${id}"&gt;OFF&lt;/span&gt;
&lt;/div&gt;
&lt;p class="note" id="rn${id}" hidden&gt;&lt;/p&gt;
&lt;div class="pair"&gt;&lt;button class="b on" data-r="${id}" data-s="1"&gt;ON&lt;/button&gt;&lt;button
class="b off" data-r="${id}" data-s="0"&gt;OFF&lt;/button&gt;&lt;/div&gt;
&lt;div class="timer"&gt;&lt;div class="trow"&gt;&lt;span&gt;Timer&lt;/span&gt;&lt;label class="sw"&gt;&lt;
input type="checkbox" id="te${id}" data-d="${id}" aria-label="Timer on or off"&gt;&lt;i&gt;&lt;/i&gt;&lt;/
label&gt;&lt;/div&gt;
&lt;div class="tin"&gt;&lt;label&gt;Turn on at&lt;input type="time" id="ton${id}" data-d="${id}"&gt;&lt;/
label&gt;&lt;label&gt;Turn off at&lt;input type="time" id="toff${id}" data-d="${id}"&gt;&lt;/label&gt;&lt;/
div&gt;
&lt;button class="b save" data-save="${id}"&gt;Save timer&lt;/button&gt;&lt;p class="info" id="ts${id}"&gt;
&lt;/p&gt;&lt;/div&gt;&lt;/article&gt;`).join('');
SENS.forEach(x=&gt;{$('s-'+x.k).innerHTML=`&lt;article class="card" id="c-${x.k}"&gt;
&lt;div class="ch"&gt;&lt;h3&gt;${x.name}&lt;/h3&gt;&lt;span class="pill" id="sp-${x.k}"&gt;ON&lt;/span&gt;
&lt;/div&gt;
&lt;div class="read"&gt;&lt;div class="ico"&gt;&lt;svg viewBox="0 0 24 24"&gt;${x.ico}&lt;/svg&gt;&lt;/div&gt;
&lt;div&gt;&lt;div class="big" id="sb-${x.k}"&gt;Not detected&lt;/div&gt;&lt;div class="sub" id="ss-${x.
k}"&gt;&lt;/div&gt;&lt;/div&gt;&lt;/div&gt;
&lt;p class="raw" id="sr-${x.k}"&gt;&lt;/p&gt;
&lt;div class="pair"&gt;&lt;button class="b on" data-sn="${x.k}" data-s="1"&gt;Sensor ON&lt;/button&gt;&lt;
button class="b off" data-sn="${x.k}" data-s="0"&gt;Sensor OFF&lt;/button&gt;&lt;/div&gt;
&lt;p class="info"&gt;${x.info}&lt;/p&gt;&lt;/article&gt;`});
/* ---------- helpers ---------- */
let tt;function toast(m){const t=$('toast');t.textContent=m;t.classList.add('show');clearTimeout(tt);
tt=setTimeout(()=&gt;t.classList.remove('show'),2600)}
async function api(path,opt){
 const c=new AbortController(),to=setTimeout(()=&gt;c.abort(),4000);
 try{const r=await fetch(path,Object.assign({signal:c.signal,cache:'no-store'},opt||{}));clearTimeout(to);
  if(r.status===401&amp;&amp;path!=='/login'){toLogin();throw new Error('auth')}
  let j={};try{j=await r.json()}catch(e){}
  return{ok:r.ok,status:r.status,j:j}}
 catch(e){clearTimeout(to);throw e}
}
async function fetchState(){const r=await api('/api/state');if(r.ok)render(r.j)}
function setLive(v){online=v;$('livebox').classList.toggle('off',!v);$('livetxt').textContent=v?'Live':'Board
offline, reconnecting'}
/* ---------- render ---------- */
function render(s){
 $('clock').textContent=s.synced?s.time:'Syncing time';
 $('ip').textContent=s.ip;$('rssi').textContent=s.rssi+' dBm';
 s.r.forEach(r=&gt;{const id=r.id;
  $('rp'+id).textContent=r.on?'ON':'OFF';$('rp'+id).className='pill'+(r.on?' lit':'');
  document.querySelector(`#r${id} [data-s="1"]`).classList.toggle('act',r.on);
  document.querySelector(`#r${id} [data-s="0"]`).classList.toggle('act',!r.on);
  const n=$('rn'+id);n.hidden=!r.auto;
  if(r.auto)n.textContent=(id===3?'PIR':'LDR')+' sensor is controlling this relay. Turn the '+(id===3?'PIR':
'LDR')+' sensor off to use the buttons and timer.';
  $('r'+id).classList.toggle('auto',r.auto);
  if(!dirty[id]){$('te'+id).checked=r.se;$('ton'+id).value=r.on_t;$('toff'+id).value=r.off_t}
  $('ts'+id).textContent=r.se?`Runs every day: on at ${r.on_t}, off at ${r.off_t}.`:'Timer is off.';
 });
 SENS.forEach(x=&gt;{const d=s[x.k],c=$('c-'+x.k),det=d.en&amp;&amp;d.det;
  $('sp-'+x.k).textContent=d.en?'ON':'OFF';$('sp-'+x.k).className='pill'+(d.en?' lit':'');
  $('sb-'+x.k).textContent=!d.en?'Sensor off':det?'Detected':'Not detected';
  $('ss-'+x.k).textContent=!d.en?'Turn the sensor on to start sensing':det?x.yes:x.no;
  $('sr-'+x.k).textContent=(x.k!=='pir'&amp;&amp;d.en)?'Sensor reading '+d.raw:'';
  c.classList.toggle('det',det&amp;&amp;x.k!=='gas');c.classList.toggle('danger',det&amp;&amp;x.k==='gas');
  c.querySelector('[data-s="1"]').classList.toggle('act',d.en);
  c.querySelector('[data-s="0"]').classList.toggle('act',!d.en);
  $('td-'+x.k).hidden=!det;
 });
}
/* ---------- polling ---------- */
async function poll(){
 if(!appOn)return;
 try{await fetchState();fails=0;if(!online)setLive(true)}
 catch(e){if(e.message!=='auth'&amp;&amp;++fails&gt;=2&amp;&amp;online)setLive(false)}
 if(appOn)timer=setTimeout(poll,1000);
}
/* ---------- screens ---------- */
function showOnly(id){['login','app'].forEach(k=&gt;$(k).hidden=(k!==id))}
function enterApp(){showOnly('app');if(!appOn){appOn=true;fails=0;setLive(true);poll()}}
function toLogin(){appOn=false;clearTimeout(timer);$('lp').value='';showOnly('login')}
$('eye').onclick=()=&gt;{const p=$('lp'),h=p.type==='password';p.type=h?'text':'password';$('eye').
textContent=h?'Hide':'Show'};
```

```
$('lf').onsubmit=async e=&gt;{
 e.preventDefault();$('le').textContent='';$('lb').disabled=true;
 try{const r=await api('/login',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'u='+encodeURIComponent($('lu').value.trim())+'&amp;p='+encodeURIComponent($('lp').value)});
  if(r.ok)enterApp();else $('le').textContent='Wrong username or password. Check them and try again.'}
 catch(x){$('le').textContent='Cannot reach the board. Check the Wi-Fi and try again.'}
 $('lb').disabled=false;
};
$('out').onclick=async()=&gt;{try{await api('/logout')}catch(e){}toLogin()};
/* ---------- tabs and buttons ---------- */
document.addEventListener('click',async e=&gt;{
 const t=e.target.closest('[data-tab],[data-r],[data-save],[data-sn],[data-cmd],[data-ir]');if(!t)return;
 try{
  if(t.dataset.tab){document.querySelectorAll('nav button').forEach(b=&gt;b.classList.toggle('sel',b===t));
   document.querySelectorAll('.panel').forEach(p=&gt;p.classList.toggle('sel',p.id==='p-'+t.dataset.tab));
   if(IRD.includes(t.dataset.tab))loadIr(t.dataset.tab);
   return}
  if(t.dataset.ir){const dev=t.dataset.ir,name=t.dataset.name;
   const r=await api(`/api/irsend?dev=${dev}&amp;name=${encodeURIComponent(name)}`);
   toast(r.ok?'Sent: '+name:'Could not send '+name);return}
  if(t.dataset.cmd){runCommand(t.dataset.cmd);return}
  if(t.dataset.r){const id=+t.dataset.r,r=await api(`/api/relay?id=${id}&amp;s=${t.dataset.s}`);
   if(r.status===409)toast(`Relay ${id} is controlled by the ${id===3?'PIR':'LDR'} sensor`);}
  if(t.dataset.sn){await api(`/api/sensor?n=${t.dataset.sn}&amp;s=${t.dataset.s}`)}
  if(t.dataset.save){const id=t.dataset.save,on=$('ton'+id).value,off=$('toff'+id).value;
   if(!on||!off){toast('Pick both an on time and an off time');return}
   await api(`/api/sched?id=${id}&amp;en=${$('te'+id).checked?1:0}&amp;on=${on}&amp;off=${off}`);dirty[
id]=false;toast('Timer saved for Relay '+id)}
  await fetchState();
 }catch(x){if(x.message!=='auth')toast('Board not reachable')}
});
document.addEventListener('input',e=&gt;{const d=e.target.dataset&amp;&amp;e.target.dataset.d;if(d)dirty[
d]=true});
/* ---------- live line under the title ---------- */
(function(){
 const cv=$('live'),cx=cv.getContext('2d'),still=matchMedia('(prefers-reduced-motion:reduce)').matches;
 const P=[[0,0],[30,0],[38,.15],[46,0],[62,0],[67,-.2],[73,.85],[79,-.45],[85,0],[108,0],[120,.2],[132,0],[
160,0]];
 let W=0,H=0,sc=0;
 function size(){const d=devicePixelRatio||1;W=cv.clientWidth;H=cv.clientHeight;cv.width=W*d;cv.height=H*d;cx.
setTransform(d,0,0,d,0,0)}
 function amp(x){x=((x%160)+160)%160;for(let i=1;i&lt;P.length;i++)if(x&lt;=P[i][0]){const a=P[i-1],b=P[i];
return a[1]+(b[1]-a[1])*(x-a[0])/(b[0]-a[0])}return 0}
 function draw(){
  if(W!==cv.clientWidth||H!==cv.clientHeight)size();
  if(W){
   cx.clearRect(0,0,W,H);
   const m=H/2+5,k=H*.4,flat=!online,col=flat?'#ff5468':'#4cc3ff';
   const g=cx.createLinearGradient(0,0,W,0);
   g.addColorStop(0,'rgba(76,195,255,0)');g.addColorStop(.65,col);g.addColorStop(1,flat?col:'#ffffff');
   cx.strokeStyle=g;cx.lineWidth=2;cx.lineJoin='round';cx.shadowColor=flat?'#ff5468':'#2b7fff';cx.
shadowBlur=10;
   cx.beginPath();let y=m;
   for(let x=0;x&lt;=W;x+=2){y=m-(flat?0:amp(x+sc))*k;if(x)cx.lineTo(x,y);else cx.moveTo(x,y)}
   cx.stroke();
   cx.fillStyle='#fff';cx.beginPath();cx.arc(W-3,y,3.5,0,7);cx.fill();
   if(!still)sc+=1.7;
  }
  requestAnimationFrame(draw);
 }
 draw();
})();
/* ---------- voice assistant ---------- */
function parseCmd(text){
 const t=' '+text.toLowerCase().replace(/[^a-z0-9 ]/g,' ')+' ';
 const a=t.match(/\b(on|off)\b/g);
 if(!a)return{err:'Say on or off with a device, for example relay 1 on.'};
 const s=a[a.length-1]==='on'?1:0;
 if(/\b(ldr|dark)\b|\bl\s*d\s*r\b/.test(t))return{s,tg:'ldr'};
 if(/\b(pir|pier|peer|motion)\b/.test(t))return{s,tg:'pir'};
 if(/\b(gas|smoke)\b/.test(t))return{s,tg:'gas'};
 const N={1:1,one:1,won:1,first:1,3:3,three:3,tree:3,third:3,4:4,four:4,fourth:4};
 const m=t.match(/relay\s+(\w+)/);
 if(m&amp;&amp;m[1]!=='on'&amp;&amp;m[1]!=='off'){const n=N[m[1]]||(m[1]==='for'?4:0);return n?{s,tg:n}:{err:
'I control relays 1, 3 and 4.'}}
 const q=t.match(/\b(1|one|first|3|three|third|4|four|fourth)\b/);
 return q?{s,tg:N[q[1]]}:{s,tg:'all'};
}
function speak(m){try{if(!('speechSynthesis' in window))return;speechSynthesis.cancel();const u=new
SpeechSynthesisUtterance(m);u.rate=1;u.pitch=1;speechSynthesis.speak(u)}catch(e){}}
function reply(m,talk){$('said').textContent=m;if(talk!==false)speak(m)}
async function runCommand(text){
 $('heard').textContent='\u201C'+text+'\u201D';
 const c=parseCmd(text);if(c.err){reply(c.err);return}
 const w=c.s?'on':'off';
 try{
  if(typeof c.tg==='string'&amp;&amp;c.tg!=='all'){await api(`/api/sensor?n=${c.tg}&amp;s=${c.s}`);await
fetchState();reply(c.tg.toUpperCase()+' sensor '+w+'.');return}
  const ids=c.tg==='all'?RL:[c.tg],blocked=[];
  for(const id of ids){const r=await api(`/api/relay?id=${id}&amp;s=${c.s}`);if(r.status===409)blocked.push(
```

```
id)}
  await fetchState();
  let msg=c.tg==='all'?`All relays ${w}.`:`Relay ${c.tg} ${w}.`;
  if(blocked.length)msg=ids.length===1?`Relay ${c.tg} is controlled by its sensor. Turn that sensor off first.
`:`Relays ${w}, except relay ${blocked.join(' and ')}, which a sensor controls.`;
  reply(msg);
 }catch(e){if(e.message!=='auth')reply('The board is not reachable.')}
}
const SR=window.SpeechRecognition||window.webkitSpeechRecognition;let rec=null,listening=false;
function micUI(on){listening=on;$('mic').classList.toggle('rec',on);$('state').textContent=on?'Listening':
'Tap the mic and speak'}
$('mic').onclick=()=&gt;{
 if(listening&amp;&amp;rec){rec.stop();return}
 if(!SR){reply('Voice needs Chrome or Edge. Type your command below instead.',false);return}
 rec=new SR();rec.lang='en-IN';rec.interimResults=true;rec.maxAlternatives=1;
 rec.onstart=()=&gt;{micUI(true);$('said').textContent='';$('heard').textContent=''};
 rec.onend=()=&gt;micUI(false);
 rec.onerror=e=&gt;{const m=e.error;reply(m==='not-allowed'||m==='service-not-allowed'?'The microphone is
blocked. Allow it in the browser, or type your command below.':m==='no-speech'?'I did not hear anything. Tap
the mic and try again.':'Voice input failed. Tap the mic to retry, or type your command.',false)};
 rec.onresult=e=&gt;{let x='';for(let i=e.resultIndex;i&lt;e.results.length;i++)x+=e.results[i][0].transcript;
  $('heard').textContent='\u201C'+x+'\u201D';if(e.results[e.results.length-1].isFinal)runCommand(x)};
 try{rec.start()}catch(x){}
};
function sendTyped(){const v=$('typed').value.trim();if(v){$('typed').value='';runCommand(v)}}
$('send').onclick=sendTyped;
$('typed').addEventListener('keydown',e=&gt;{if(e.key==='Enter')sendTyped()});
/* ---------- start: 2 second entry flash, then login or app ---------- */
(async function(){
 const wait=new Promise(r=&gt;setTimeout(r,2000));
 const check=api('/api/state').then(r=&gt;r.ok).catch(()=&gt;false);
 const [,ok]=await Promise.all([wait,check]);
 $('splash').classList.add('out');
 if(ok)enterApp();else showOnly('login');
 setTimeout(()=&gt;{$('splash').hidden=true},500);
})();
&lt;/script&gt;
&lt;/body&gt;&lt;/html&gt;)rawliteral";
void handleRoot() {
  server.sendHeader("Cache-Control", "no-cache");
  server.send_P(200, "text/html", PAGE);
}
void handleFavicon() {
  server.send(204);
}
// ========================================================================
//  Setup / Loop
// ========================================================================
unsigned long lastSensor = 0, lastSched = 0, lastWifi = 0, lastPrint = 0;
void setup() {
  Serial.begin(115200);
  // ---------- Load saved settings (FIX: these were never created/loaded) ----------
  cfgMux = xSemaphoreCreateMutex();
  tgQ = xQueueCreate(4, sizeof(TgMsg));
  xTaskCreate(tgTask, "tg", 8192, nullptr, 1, nullptr);
  prefs.begin("smartlife", false);
#if RESET_PASSWORD
  prefs.remove("pw");
#endif
  loginPass = prefs.getString("pw", LOGIN_PASS);   // first time = "smartlife"
  tgToken   = prefs.getString("tgt", "");
  tgIds     = prefs.getString("tgi", "");
  tgOn      = prefs.getBool("tgo", true);
  acTemp    = prefs.getInt("actemp", 24);
  prefs.end();
  pinMode(RELAY1, OUTPUT);
  pinMode(RELAY3, OUTPUT);
  pinMode(RELAY4, OUTPUT);
  // Start with all relays OFF
  applyRelays();
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
  irSend.begin();
  irLoadDevice(DEV_AC);
  irLoadDevice(DEV_TV);
  irLoadDevice(DEV_PROJ);
  loadSched();
  // ---------- WiFi (kept awake so the page answers fast) ----------
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);
  WiFi.setAutoReconnect(true);
#if USE_STATIC_IP
  WiFi.config(local_IP, gateway, subnet, dns1);
#endif
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected! ESP32 IP address: ");
  Serial.println(WiFi.localIP());
  configTime(GMT_OFFSET_SEC, 0, "pool.ntp.org", "time.google.com");
  // ---------- Routes ----------
  const char *hdrs[] = {"Cookie"};
  server.collectHeaders(hdrs, 1);
  server.on("/", HTTP_GET, handleRoot);
  server.on("/favicon.ico", HTTP_GET, handleFavicon);
  server.on("/ir", HTTP_GET, handleIrPage);
  server.on("/api/irlist", HTTP_GET, handleIrList);
  server.on("/api/irsend", HTTP_GET, handleIrSend);
  server.on("/api/actemp", HTTP_GET, handleAcTemp);
  server.on("/login", HTTP_POST, handleLogin);
  server.on("/logout", HTTP_GET, handleLogout);
  server.on("/api/state", HTTP_GET, handleState);
  server.on("/api/relay", HTTP_GET, handleRelay);
  server.on("/api/sched", HTTP_GET, handleSched);
  server.on("/api/sensor", HTTP_GET, handleSensor);
  server.on("/api/log", HTTP_GET, handleLog);
  server.on("/api/logclear", HTTP_GET, handleLogClear);
  server.on("/api/tg", HTTP_GET, handleTgGet);
  server.on("/api/tgset", HTTP_POST, handleTgSet);
  server.on("/api/tgtest", HTTP_GET, handleTgTest);
  server.on("/api/passwd", HTTP_POST, handlePasswd);
  server.onNotFound(handleNotFound);
  server.begin();
  Serial.print("Smart Life is live. Open http://");
  Serial.println(WiFi.localIP());
  Serial.print("IR remotes: open http://");
  Serial.print(WiFi.localIP());
  Serial.println("/ir  |  type 'help' here for pairing commands (pair ac/tv/proj &lt;name&gt;).");
}
void loop() {
  server.handleClient();
  irTick();
  unsigned long now = millis();
  // sensors every 0.2 sec (same rate as your original sketches)
  if (now - lastSensor &gt;= 200) {
    lastSensor = now;
    pirTick();
    ldrTick();
    gasTick();
    applyRelays();
    applyOutputs();
  }
  // timers checked every second
  if (now - lastSched &gt;= 1000) {
    lastSched = now;
    scheduleTick();
  }
  // calibration values in Serial Monitor once a second
  if (now - lastPrint &gt;= 1000) {
    lastPrint = now;
    Serial.printf("LDR raw %d state %d | GAS raw %d state %d | PIR %d\n", ldrRaw, ldrState, gasRaw, gasState,
holding ? 1 : 0);
  }
  // keep the connection alive: rejoin WiFi if it ever drops
  if (WiFi.status() != WL_CONNECTED &amp;&amp; now - lastWifi &gt;= 10000) {
    lastWifi = now;
    WiFi.reconnect();
  }
  delay(1);
}
```